



Reverse a Sub-list (medium)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
 - Code
- Time complexity
- Space complexity
- Similar Questions

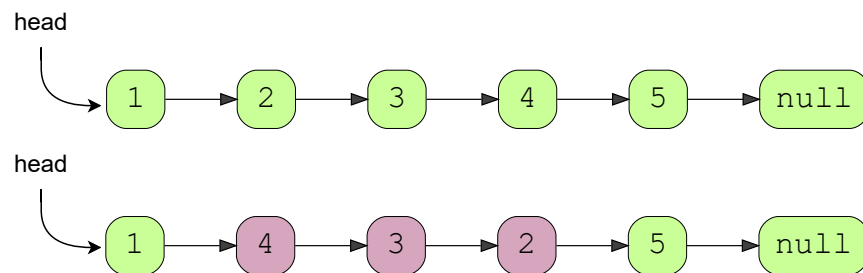
Problem Statement

Given the head of a LinkedList and two positions 'p' and 'q', reverse the LinkedList from position 'p' to 'q'.

Example:

Original List:

p=2, q=4



Try it yourself

Try solving this question here:



```
1 import java.util.*;
2
3 class ListNode {
4     int value = 0;
5     ListNode next;
6
7     ListNode(int value) {
8         this.value = value;
9     }
10 }
11
12 class ReverseSubList {
```



```

13
14 public static ListNode reverse(ListNode head, int p, int q) {
15     // TODO: Write your code here
16     return head;
17 }
18
19 public static void main(String[] args) {
20     ListNode head = new ListNode(1);
21     head.next = new ListNode(2);
22     head.next.next = new ListNode(3);
23     head.next.next.next = new ListNode(4);
24     head.next.next.next.next = new ListNode(5);
25
26     ListNode result = ReverseSubList.reverse(head, 2, 4);
27     System.out.print("Nodes of the reversed LinkedList are: ");
28     while (result != null) {
29         System.out.print(result.value + " ");
30         result = result.next;
31     }
32 }
33 }

```

Run

Save

Reset



Solution

The problem follows the **In-place Reversal of a LinkedList** pattern. We can use a similar approach as discussed in [Reverse a LinkedList](#). Here are the steps we need to follow:

1. Skip the first $p-1$ nodes, to reach the node at position p .
2. Remember the node at position $p-1$ to be used later to connect with the reversed sub-list.
3. Next, reverse the nodes from p to q using the same approach discussed in [Reverse a LinkedList](#).
4. Connect the $p-1$ and $q+1$ nodes to the reversed sub-list.

Code

Here is what our algorithm will look like:

Java

Python3

C++

JS

```

1  import java.util.*;
2
3  class ListNode {
4      int value = 0;
5      ListNode next;
6
7      ListNode(int value) {
8          this.value = value;
9      }
10 }
11
12 class ReverseSubList {
13
14     public static ListNode reverse(ListNode head, int p, int q) {
15         if (p == q)
16             return head;
17
18         // after skipping 'p-1' nodes, current will point to 'p'th node
19         ListNode current = head, previous = null;
20         for (int i = 0; current != null && i < p - 1; ++i) {

```



```

21     previous = current;
22     current = current.next;
23 }
24
25 // we are interested in three parts of the LinkedList, part before index 'p', part between 'p' and
26 // 'q', and the part after index 'q'
27 ListNode lastNodeOfFirstPart = previous; // points to the node at index 'p-1'
28 // after reversing the LinkedList 'current' will become the last node of the sub-list
29 ListNode lastNodeOfSubList = current;
30 ListNode next = null; // will be used to temporarily store the next node
31 // reverse nodes between 'p' and 'q'
32 for (int i = 0; current != null && i < q - p + 1; i++) {
33     next = current.next;
34     current.next = previous;
35     previous = current;
36     current = next;
37 }
38
39 // connect with the first part
40 if (lastNodeOfFirstPart != null)
41     lastNodeOfFirstPart.next = previous; // 'previous' is now the first node of the sub-list
42 else // this means p == 1 i.e., we are changing the first node (head) of the LinkedList
43     head = previous;
44
45 // connect with the last part
46 lastNodeOfSubList.next = current;
47
48 return head;
49 }
50
51 public static void main(String[] args) {
52     ListNode head = new ListNode(1);
53     head.next = new ListNode(2);
54     head.next.next = new ListNode(3);
55     head.next.next.next = new ListNode(4);
56     head.next.next.next.next = new ListNode(5);
57
58     ListNode result = ReverseSubList.reverse(head, 2, 4);
59     System.out.print("Nodes of the reversed LinkedList are: ");
60     while (result != null) {
61         System.out.print(result.value + " ");
62         result = result.next;
63     }
64 }
65 }

```

Run

Save

Reset



Time complexity

The time complexity of our algorithm will be $O(N)$ where 'N' is the total number of nodes in the LinkedList.

Space complexity

We only used constant space, therefore, the space complexity of our algorithm is $O(1)$.

Similar Questions

Problem 1: Reverse the first 'k' elements of a given LinkedList.



Solution: This problem can be easily converted to our parent problem; to reverse the first 'k' nodes of the list, we need to pass `p=1` and `q=k`.

Problem 2: Given a LinkedList with 'n' nodes, reverse it based on its size in the following way:



1. If 'n' is even, reverse the list in a group of $n/2$ nodes.
2. If n is odd, keep the middle node as it is, reverse the first ' $n/2$ ' nodes and reverse the last ' $n/2$ ' nodes.

Solution: When 'n' is even we can perform the following steps:

1. Reverse first ' $n/2$ ' nodes: `head = reverse(head, 1, n/2)`
2. Reverse last ' $n/2$ ' nodes: `head = reverse(head, n/2 + 1, n)`

When 'n' is odd, our algorithm will look like:

1. `head = reverse(head, 1, n/2)`
2. `head = reverse(head, n/2 + 2, n)`

Please note the function call in the second step. We're skipping two elements as we will be skipping the middle element.

[< Back](#)

Reverse a LinkedList (easy)

[✔ Completed](#)[Next >](#)

Reverse every K-element Sub-list (medium)

